

Extract Numerical & Categorical Features from the Ticket Column

Goal: Split the `Ticket` column into:

- `ticket_num` → Numerical part of the ticket
- `ticket_cat` → Prefix/category of the ticket

Extract Numerical Part

```
```python id="8klh4n" df['ticket_num'] = df['Ticket'].apply( lambda s: s.split()[-1] )
```

```
df['ticket_num'] = pd.to_numeric(df['ticket_num'], errors='coerce', downcast='integer')
```

### Keywords

- `apply()` → Apply a function to each row.
- `lambda` → Anonymous (one-line) function.
- `split()` → Split a string into a list using whitespace.
- `[-1]` → Select the last element of the list.
- `pd.to_numeric()` → Convert values to numeric type.
- `errors='coerce'` → Invalid values become `NaN`.
- `downcast='integer'` → Reduce memory usage.

### Example

```
```python id="exl4sm"
'PC 17599'      → 17599
'A/5 21171'     → 21171
'347082'        → 347082
```

Extract Categorical Part

```
```python id="w40w2q" df['ticket_cat'] = df['Ticket'].apply( lambda s: s.split()[0] )
```

```
df['ticket_cat'] = np.where(df['ticket_cat'].str.isdigit(), np.nan, df['ticket_cat'])
```

### Keywords

- `[0]` → Select the first element of the list.
- `str.isdigit()` → Check whether a string contains only digits.
- `np.where(condition, x, y)` → Vectorized if-else operation.

### Logic

- If the first part contains only digits → Replace with `NaN`
- Otherwise → Keep the ticket prefix

### Example

```
```text id="72g44r"
'PC 17599'      → PC
'A/5 21171'     → A/5
'347082'        → NaN
```

Quick Revision

- `split()[0]` → First token
- `split()[-1]` → Last token
- `apply(lambda)` → Apply custom logic row-wise
- `isdigit()` → Check for numeric strings
- `to_numeric()` → Convert to numbers
- `coerce` → Invalid values → `NaN`
- `np.where()` → If-else for arrays

Use Case

Extract useful features from text columns before machine learning or feature engineering.